

Verification

1,117 unit and component tests across 71 files, plus Playwright end-to-end, accessibility, responsive, and visual suites. Counts are cheap, so this document is about the tests that catch things a normal suite does not.

The class of bug this codebase kept producing

Almost every serious defect found in Continuum failed as an empty array or a missing rule, never as an exception:

- A view omitted a field, so four panels rendered permanently empty.
- A retrieval leg missed its deadline and returned `[]`, indistinguishable from an empty library.
- A component rendered classes whose stylesheet was never imported, so the concept map on the goal page displayed as unstyled inline text.
- A utility class lost the cascade to a later rule of equal specificity, so `.mobile-only` never hid anything at any width, since the day it was written.

None of these throw. All of them render as something that looks intentional. So the assertions below are written to fail on absence, not on error.

Contract tests

`tests/view-contract.test.ts` asserts:

- Every screen reads only fields its view returns.
- A concept carries its real title rather than an id to be humanised.
- A practice set's best score comes from an aggregate, not from re-reading attempts.
- The passage-only lexical query exists, and it is what the assistant's fallback calls.
- `searchResearch` still orders passages last, which is *why* it must not be used for passage retrieval. The test asserts the property that makes the other test necessary.
- Read endpoints do not statically import write-path dependencies, and `sharp` stays behind a lazy loader.
- Every endpoint has a caller and every control has an endpoint.
- Each component owning a namespaced class family imports the stylesheet defining it, counting `globals.css` and the kit as always loaded because the root layout pulls them in.

That last assertion was added after the concept-map defect, and it was verified by removing the import and confirming the test fails.

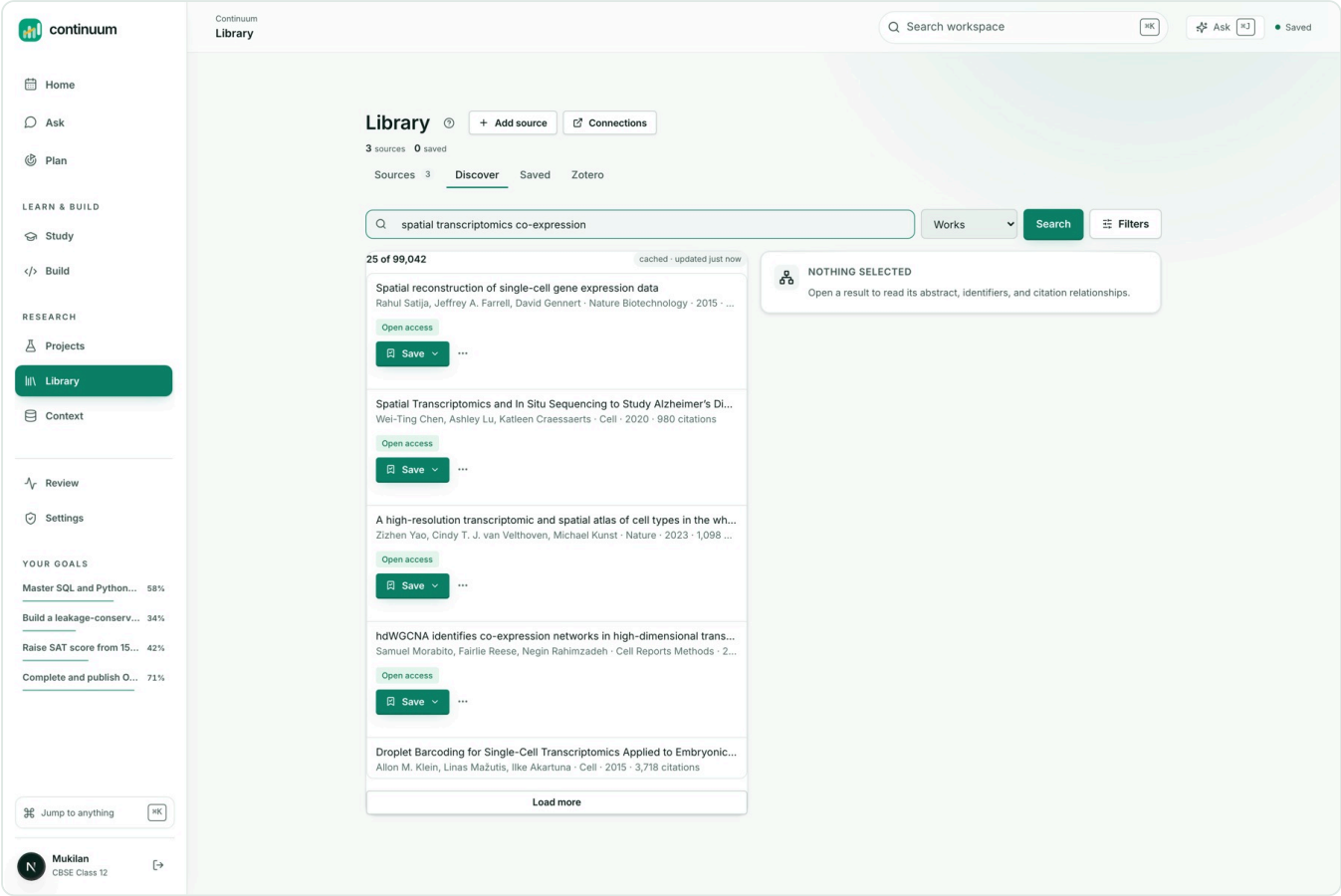
Design-system tests

`tests/design-tokens.test.ts` holds `globals.css` under a 600-line ceiling and asserts every co-located stylesheet composes tokens rather than literal colours. It also asserts that every token a module references is actually defined somewhere, because an undefined custom property has no error state: the element silently inherits.

That check was written after `--font-display` turned out to be referenced by the assistant's welcome heading and defined nowhere.

Seed integrity

`tests/demo-seed.test.ts` asserts the seeded demo is internally consistent, for example that the question-bank set scoring lowest is the one whose concept has the most lapses, and that every `sourceChunkId` on a question points at a chunk that exists. A demo whose numbers disagree with each other is worse than no demo.



Discover, running a live OpenAlex query. Result rows were the subject of four layout repairs before they held a title, byline, badge and an unclipped Save.

Contrast measured, not eyeballed

A layout auditor runs in-page across every route at 1920, 1440, 1280, 1100, 768 and 390 px, computing WCAG contrast for every text node against its resolved backdrop, plus overflow, clipping, overlap, and pointer-target size.

It found two systemic failures that manual review had missed for months:

- `--ink-3` carries nearly every caption, timestamp and helper line in the product and failed AA on all four surfaces it sits on: 4.28:1 on white, 3.95 on raised, 3.70 on sunken.
- `--brand-hover` was *lighter* than `--brand`, so hovering a primary button dropped its white label from 4.30:1 to 3.30:1. The most important control on each screen got harder to read exactly as you reached for it.

Both are now pinned with the measured ratio written beside the value:

```
--brand:          #0c8168; /* white on this: 4.82:1 */
--brand-hover:    #0a7660; /* white on this: 5.57:1 */
--brand-strong:   #0a6b55; /* white on this: 6.47:1 */
```

The auditor itself needed two fixes before its output could be trusted: it could not parse Chrome's `color(srgb ...)` serialization, which turned white into near-black and produced fake failures, and it ignored background gradients, which flagged every element on the dark hero card. Both were corrected before any token was changed.

Verification by deliberate breakage

For the fixes that matter, the test was confirmed to fail without the fix. That is a small discipline with a large payoff: a passing test that would also pass against the bug is not protection, it is decoration.

What this does not prove

The discovery rate has not flattened. In the most recent pass alone, driving the app rather than reading it surfaced a production 500 on the Library, a flagship screen rendering unstyled, a utility class that never worked, two dead capabilities, and an assistant answer that named one of its own JSON keys. Two of those the user found before the tests did.

So the honest claim is not "this is correct". It is: every defect found now has a test that fails without its fix, and the tests are written to fail on silence.